

Нотация

Пусть последовательность имеет длину T . Для позиции t используются:

- q_t — query текущего токена;
- k_t — key;
- v_t — value;
- $q_t, k_t \in \mathbb{R}^{d_k}$;
- $v_t \in \mathbb{R}^{d_v}$;
- S_t — состояние, агрегирующее историю;
- M — causal mask;
- $v_t k_t^\top$ — outer product, то есть одна запись в матричное состояние.

Outer product $v_t k_t^\top$ имеет форму $d_v \times d_k$. Поэтому состояние S_t также имеет фиксированную форму $d_v \times d_k$, независимо от числа уже обработанных токенов.

Три вычислительные формы, используемые далее:

- **parallel** — все попарные взаимодействия представлены явной матрицей;
- **recurrent** — последовательность обрабатывается через цепочку состояний фиксированной формы;
- **chunkwise** — внутри небольших блоков используется parallel-вычисление, а между блоками передаётся recurrent state.

Важно различать **математический оператор** и **алгоритм его вычисления**. Один оператор может иметь несколько точно эквивалентных вычислительных форм с разной стоимостью.

Стоимость модели как многомерная величина

Стоимость архитектуры нельзя адекватно описать только FLOPs или одной асимптотикой. В лекции используется вектор стоимости из девяти координат:

$$C = (M_{\text{weights}}, M_{\text{persistent}}, M_{\text{workspace}}, B_{\text{HBM}}, F, D_{\text{seq}}, N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}).$$

Для темы занятия существенны первые шесть координат.

M_{weights} — память весов

Память, необходимая для параметров модели. В отличие от памяти контекста, сама по себе не растёт с длиной последовательности.

$M_{\text{persistent}}$ — persistent memory

Данные, которые должны сохраняться между последовательными шагами вычисления.

Для обычного autoregressive attention основным примером является KV-cache. Для recurrent linear attention — состояние S_t фиксированной формы.

$M_{\text{workspace}}$ — рабочая память

Временные промежуточные объекты, необходимые только во время одного вычисления.

Например, полная матрица попарных взаимодействий размером $T \times T$ относится к workspace, а не к persistent state. Её можно не материализовывать целиком, вычисляя взаимодействия блоками.

B_{HBM} — traffic через память

Объём данных, который приходится переносить между HBM и вычислительными блоками GPU.

Количество арифметических операций и объём memory traffic — разные характеристики. На decode длинного контекста именно чтение памяти может оказаться главным ограничением даже при относительно небольшом количестве арифметики.

F — FLOPs

Число арифметических операций.

Меньшее число FLOPs не гарантирует меньшую latency: большое матричное умножение может использовать GPU эффективнее, чем множество маленьких последовательных операций.

D_{seq} — sequential depth

Число вычислительных стадий, которые принципиально не могут выполняться одновременно.

Если вычисление позиции $t + 1$ зависит от результата позиции t , добавление вычислительных блоков само по себе не устраняет эту зависимость.

$N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}$

Эти координаты зарезервированы для следующих частей курса. Для N_{FE} используется смысл числа вызовов модели, для $N_{\text{processed}}$ — количества обработанных токенов. Семантика B_{max} в рамках этого занятия не раскрывается.

Вместимость памяти и скорость — разные вопросы

Условие размещения модели на GPU можно записать как

$$M_{\text{weights}} + M_{\text{persistent}} + M_{\text{workspace}} + M_{\text{runtime}} \leq M_{\text{GPU}}.$$

Оно отвечает на вопрос, **помещается ли вычисление в память**, но ничего непосредственно не говорит о latency или memory traffic.

Архитектурная оптимизация почти всегда является разменом между координатами: можно уменьшить persistent memory ценой дополнительной последовательности, уменьшить FLOPs ценой более неудобных операций для GPU и т. д.

Поэтому утверждения вида «архитектура линейная» или «в шесть раз быстрее» недостаточны без уточнения:

- какая координата стоимости измеряется;
- при какой длине последовательности;
- в каком режиме — training, prefill или decode;
- на каком железе;
- какой ценой достигнуто улучшение.

KV-cache

Зачем нужен KV-cache

При autoregressive decode каждый новый токен формирует новый query и должен взаимодействовать с представлениями предыдущих токенов.

Keys и values прошлого можно вычислять повторно на каждом шаге, но это означает постоянное повторение уже выполненной работы. Поэтому после первого вычисления K и V сохраняются и

переиспользуются.

KV-cache является **persistent state**, потому что должен переживать один decode step и использоваться на следующих.

При добавлении каждого нового токена cache увеличивается.

Сколько чисел добавляет один токен

Для одного токена в одном слое необходимо сохранить key и value для каждой distinct KV-head.

Количество scalars на токен:

$$2n_{\text{layers}}n_{\text{KV}}d_{\text{head}}.$$

Множитель **2** появляется из-за двух объектов — K и V .

Для префикса длины L :

$$2Ln_{\text{layers}}n_{\text{KV}}d_{\text{head}}.$$

Из этих множителей именно L растёт во время decode; остальные задаются архитектурой.

KV-cache в байтах

Если одно число занимает s_{dtype} байт:

$$M_{\text{KV}} = Ln_{\text{layers}}n_{\text{KV}}d_{\text{head}}s_{\text{dtype}} \times 2.$$

Для B одновременно обслуживаемых последовательностей:

$$M_{\text{KV}} = BLn_{\text{layers}}n_{\text{KV}}d_{\text{head}}s_{\text{dtype}} \times 2.$$

Здесь B — число параллельных последовательностей, а не длина контекста.

Persistent memory и workspace attention

Два разных эффекта часто ошибочно объединяют под выражением «память attention»:

1. Матрица попарных взаимодействий $T \times T$ — временный workspace при обработке всей последовательности.
2. KV-cache — persistent memory между decode steps.

Workspace можно вычислять плитками и не хранить всю $T \times T$ матрицу одновременно.

KV-cache таким способом устранить нельзя: прошлые keys и values понадобятся снова после завершения текущего шага.

Поэтому квадратичный workspace и линейно растущий persistent cache — принципиально разные проблемы.

МНА, GQA и MQA

Размер KV-cache линейно зависит от числа distinct KV-heads:

$$M_{KV} \propto n_{KV}.$$

Multi-Head Attention — МНА

Каждая query-head имеет собственные key/value heads.

В примере:

- 32 query-heads;
- 32 KV-heads.

То есть $n_{KV} = 32$.

Grouped-Query Attention — GQA

Несколько query-heads совместно используют одну пару key/value heads.

Например:

- 32 query-heads;
- 8 KV-heads;
- по 4 query-heads на одну KV-head.

Query для каждой головы остаётся собственным, но keys и values внутри группы общие.

Multi-Query Attention — MQA

Крайний случай sharing:

- много query-heads;
- одна KV-head на весь слой.

То есть $n_{KV} = 1$.

Численный пример для контекста 100 000 токенов

Параметры:

- $L = 100000$;
- $n_{\text{layers}} = 32$;
- $d_{\text{head}} = 128$;
- $s_{\text{dtype}} = 2$ байта;
- $B = 1$.

MHA: 32 KV-heads

$$M_{KV} = 100000 \times 32 \times 32 \times 128 \times 2 \times 2 = 52\,428\,800\,000 \text{ bytes.}$$

Это примерно 52.43 GB или 48.83 GiB.

GQA: 8 KV-heads

$$M_{KV} = 100000 \times 32 \times 8 \times 128 \times 2 \times 2 = 13\,107\,200\,000 \text{ bytes.}$$

Это примерно 13.11 GB или 12.21 GiB.

MQA: одна KV-head

$$M_{KV} = 100000 \times 32 \times 1 \times 128 \times 2 \times 2 = 1\,638\,400\,000 \text{ bytes.}$$

Это примерно 1.64 GB или 1.53 GiB.

При одинаковой заявленной длине контекста разница между MHA и MQA в этом примере составляет 32 раза.

GB и GiB различаются базой:

- $1\text{ GB} = 10^9$ байт;
- $1\text{ GiB} = 2^{30}$ байт.

Что означает уменьшение числа KV-heads

GQA и MQA — не способы «сжать» KV-cache уже существующей модели. Число KV-heads является частью архитектуры.

Модель с 8 KV-heads и модель с 32 KV-heads — разные модели. Простое изменение конфигурационного параметра уже обученной MHA-модели не превращает её в эквивалентную GQA-модель.

Уменьшение n_{KV} также означает sharing памяти между несколькими query-heads. Поэтому влияние на качество является эмпирическим вопросом и не выводится из формулы памяти.

KV-cache и количество одновременно обслуживаемых последовательностей

Persistent context memory конкурирует за GPU memory с весами и другими последовательностями.

Поэтому уменьшение KV-cache влияет не только на возможность увеличить context length, но и на максимальный batch/concurrency: если cache одной последовательности уменьшился в четыре раза, при прочих равных потенциально можно разместить больше одновременно активных контекстов.

Почему длинный softmax decode ограничивается не только FLOPs

При каждом новом decode step query должен взаимодействовать со всеми накопленными keys.

Следовательно, объём чтения KV-cache на один новый токен растёт вместе с длиной префикса.

Даже если арифметики сравнительно немного, десятки гигабайт чтения на каждый новый токен могут сделать вычисление memory-bandwidth bound: вычислительные блоки способны выполнить операции быстрее, чем память успевает подать данные.

Поэтому при анализе decode длинного контекста важно отдельно смотреть на:

- arithmetic complexity;
- размер persistent cache;
- объём чтения cache на один токен;
- доступную пропускную способность памяти.

Уменьшение FLOPs само по себе не объясняет ускорение memory-bound decode.

Softmax attention и причина хранения истории поэлементно

Для исходных query и key обозначим $\hat{q}$ и $\hat{k}$. Softmax kernel имеет вид

$$\kappa_{\text{sm}}(\hat{q}, \hat{k}) = \exp\left(\frac{\hat{q}^\top \hat{k}}{\sqrt{d_h}}\right).$$

Нормированный causal attention:

$$y_t = \frac{\sum_{i \leq t} \kappa_{\text{sm}}(\hat{q}_t, \hat{k}_i) v_i}{\sum_{i \leq t} \kappa_{\text{sm}}(\hat{q}_t, \hat{k}_i)}.$$

Для конкретного query веса всех прошлых values определяются совместным взаимодействием этого query с каждым key.

Чтобы историю можно было агрегировать заранее в конечномерный state, interaction желательно представить как произведение:

- части, зависящей только от query;
- части, зависящей только от key.

В используемой здесь конечномерной постановке softmax interaction такой точной факторизации не имеет. Поэтому contribution прошлого key нельзя окончательно определить до появления текущего query.

Отсюда необходимость сохранять прошлые keys отдельно.

Переход к факторизуемому kernel

Вместо softmax kernel вводится feature map

$$\phi : \mathbb{R}^{d_h} \rightarrow \mathbb{R}^{d_k}.$$

Преобразованные query и key:

$$q = \phi(\hat{q}), \quad k = \phi(\hat{k}).$$

Kernel задаётся скалярным произведением признаков:

$$\kappa(\hat{q}, \hat{k}) = \phi(\hat{q})^\top \phi(\hat{k}).$$

Это позволяет перегруппировать суммы благодаря ассоциативности матричного умножения.

Ключевой смысл: **замена softmax kernel на факторизуемый kernel меняет математический оператор.**

Это не другой алгоритм точного вычисления прежнего softmax attention.

Следовательно:

- значения output в общем случае другие;
- модель должна обучаться с новым оператором;
- отсутствие растущего KV-cache достигается изменением функции, а не бесплатной оптимизацией той же функции.

Замена оператора и приближение softmax — разные подходы

Есть два разных класса методов:

1. **Kernel replacement** — с самого начала определяется другой факторизуемый kernel. Вычисление может быть точным относительно нового оператора.
2. **Softmax approximation** — исходный softmax kernel приближается, например random features. Тогда возникает approximation error относительно softmax.

Полезный вопрос при чтении статьи: пытается ли метод вычислить новый оператор или заявляет приближение исходного softmax.

Неотрицательные feature maps

Если взять обычное скалярное произведение без дополнительного преобразования, similarity может быть отрицательной.

Для нормированного linear attention это создаёт проблему: положительные и отрицательные contributions способны взаимно сокращаться, а знаменатель — приблизиться к нулю или пройти через него.

Поэтому часто используются feature maps с неотрицательными компонентами. Типичный класс преобразований строится так, чтобы $\phi(x) \geq 0$ покомпонентно.

Неотрицательность важна прежде всего для численной устойчивости нормировки.

При этом возникает содержательный размен. Экспонента softmax резко усиливает разницу между более и менее похожими keys. Простые факторизуемые kernels обычно создают более сглаженное распределение вкладов. Это уже отличие самого оператора, а не его реализации.

Базовый causal linear operator

После feature map используются $q_t, k_t \in \mathbb{R}^{d_k}$ и $v_t \in \mathbb{R}^{d_v}$.

Базовый ненормированный causal linear operator:

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

Это та функция, для которой выводятся parallel, recurrent и chunkwise формы.

Parallel form

Causal mask:

$$M_{ti} = \begin{cases} 1, & i \leq t, \\ 0, & i > t. \end{cases}$$

Пусть

- $Q \in \mathbb{R}^{T \times d_k}$;
- $K \in \mathbb{R}^{T \times d_k}$;
- $V \in \mathbb{R}^{T \times d_v}$;
- $M \in \mathbb{R}^{T \times T}$.

Тогда весь оператор можно вычислить сразу:

$$Y = ((QK^\top) \odot M)V.$$

Здесь:

1. $QK^\top$ создаёт все query-key interactions;
2. causal mask удаляет взаимодействия с будущими позициями;
3. умножение на V формирует outputs.

Parallel form делает pairwise interactions явными и хорошо соответствует большим матричным операциям GPU.

Цена — объект размером $T \times T$ и квадратичная по T арифметика.

Переход от суммы к matrix state

Исходный оператор:

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

Скаляр $q_t^\top k_i$ можно переставить относительно v_i :

$$y_t = \left(\sum_{i=1}^t v_i k_i^\top \right) q_t.$$

Вводится состояние:

$$S_t = \sum_{i=1}^t v_i k_i^\top.$$

Его форма:

$$S_t \in \mathbb{R}^{d_v \times d_k}.$$

Главное свойство: размеры S_t зависят от d_v и d_k , но **не зависят от t** .

История длины 10, 1000 или 100 000 токенов представлена матрицей одной и той же формы.

Recurrent form

Сумму для S_t можно разложить на накопленное прошлое и текущую запись:

$$S_t = \sum_{i=1}^{t-1} v_i k_i^\top + v_t k_t^\top.$$

Следовательно:

$$S_t = S_{t-1} + v_t k_t^\top.$$

Чтение:

$$y_t = S_t q_t.$$

Полный recurrent step:

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

Для каждого нового токена нужны:

- один outer product;
- одно сложение с состоянием;
- одно matrix-vector multiplication.

Стоимость одного шага и размер persistent state не зависят от длины уже обработанного контекста.

Inclusive causality: сначала write, затем read

Оператор с условием $i \leq t$ включает текущий токен в собственный context.

Поэтому recurrent execution должна выполняться в порядке:

1. добавить $v_t k_t^\top$ в состояние;
2. прочитать $S_t q_t$.

То есть:

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

В parallel form то же решение кодируется включённой диагональю causal mask.

Один и тот же семантический выбор проявляется по-разному:

- в parallel form — диагональ маски;
- в recurrent form — порядок read/write;
- в chunkwise form — causal mask внутри блока и момент включения блока в carried state.

Strict causality — другой оператор

Если сначала выполнить read, а затем write, текущий токен не увидит себя.

Получается строгая причинность:

$$\tilde{y}_t = \sum_{i < t} (q_t^\top k_i) v_i.$$

Она отличается от inclusive operator ровно собственным contribution текущего токена:

$$y_t - \tilde{y}_t = (q_t^\top k_t) v_t.$$

При $T = 1$ различие максимально очевидно:

$$y_1 = (q_1^\top k_1) v_1, \quad \tilde{y}_1 = 0.$$

Обе версии математически осмысленны, но это **разные операторы**. Ошибка на одну диагональ маски — не мелкая численная погрешность.

Состояние с ненулевым S_0

Начальное состояние не обязано быть нулевым.

В общем случае:

$$S_t = S_0 + \sum_{i=1}^t v_i k_i^\top.$$

Это позволяет заранее обработать общий prefix и использовать полученный state как начальное состояние для следующих вычислений.

Например, одна и та же системная инструкция для множества запросов может быть представлена заранее вычисленным state.

Отличие от обычного prefix KV-cache состоит в форме объекта:

- KV prefix хранит отдельные token-indexed keys и values;
- linear state хранит один объект фиксированной формы.

Fixed-shape state не является «сжатым KV-cache»

KV-cache сохраняет каждый прошлый key/value отдельно. Поэтому конкретная запись остаётся доступной как самостоятельный объект.

В state

$$S_t = \sum_{i=1}^t v_i k_i^\top$$

отдельные contributions складываются.

После суммирования в общем случае невозможно восстановить исходные $v_i k_i^\top$ по отдельности.

Простейший одномерный пример:

- первая запись даёт $5 \cdot 2 = 10$;
- вторая — $7 \cdot 4 = 28$;
- состояние равно **38**.

По числу **38** невозможно определить, были ли слагаемые $10 + 28$, $19 + 19$ или любая другая подходящая пара.

В многомерном state возможностей разделять записи значительно больше, но фундаментальный принцип остаётся тем же: множество записей superpose в одном объекте.

Ёмкость fixed state

При $d_k = d_v = 128$ state одной головы содержит

$$d_k d_v = 128 \cdot 128 = 16384$$

числа независимо от длины последовательности.

KV representation для 100 000 токенов при тех же размерностях содержит на одну голову

$$L(d_k + d_v) = 100000(128 + 128) = 25\,600\,000$$

чисел.

Отношение:

$$\frac{L(d_k + d_v)}{d_k d_v} = \frac{100000(128 + 128)}{128 \cdot 128} = 1562.5.$$

То есть размер context representation уменьшается примерно в полторы тысячи раз.

Но это сравнение касается только соответствующей координаты persistent context memory. Оно не означает уменьшение полной стоимости модели в 1500 раз.

И оно не означает сохранение всей информации.

Interference в fixed state

Чтение state можно разложить обратно:

$$S_t q_t = \sum_{i=1}^t v_i (k_i^\top q_t).$$

Если query должен извлечь contribution некоторой записи j , output содержит не только её:

$$y_t = (k_j^\top q_t) v_j + \sum_{i \neq j} (k_i^\top q_t) v_i.$$

Второе слагаемое представляет interference остальных записей.

Если другие keys почти ортогональны query, их примесь мала. Если многие keys имеют похожее направление, cross-talk увеличивается.

При накоплении большого числа записей различные outer products неизбежно накладываются друг на друга.

Поэтому:

- fixed shape означает постоянный объём;
- fixed shape не означает бесконечную информационную ёмкость;
- fixed state не гарантирует сохранение каждого прошлого факта;
- постоянная стоимость достигается ценой возможного interference и forgetting.

Нормированный linear attention

Ненормированный operator суммирует contributions. Тогда масштаб output может увеличиваться с числом накопленных записей.

Если kernel факторизован:

$$\kappa(\hat{q}_t, \hat{k}_i) = q_t^\top k_i,$$

то нормированный output можно записать как

$$y_t = \frac{\sum_{i \leq t} (q_t^\top k_i) v_i}{\sum_{i \leq t} q_t^\top k_i}.$$

Числитель уже представлен matrix state:

$$S_t = \sum_{i \leq t} v_i k_i^\top.$$

Для знаменателя достаточно дополнительно накапливать вектор

$$z_t = \sum_{i \leq t} k_i.$$

Его recurrence:

$$z_t = z_{t-1} + k_t.$$

Тогда нормированный read принимает вид

$$y_t = \frac{S_t q_t}{z_t^\top q_t + \epsilon}.$$

Малое ϵ защищает от деления на очень маленькое число.

Таким образом, нормированный state состоит из:

- матрицы $S_t \in \mathbb{R}^{d_v \times d_k}$ для числителя;
- вектора $z_t \in \mathbb{R}^{d_k}$ для знаменателя.

Дополнительный вектор намного меньше основной матрицы и не меняет основного вывода о fixed-shape memory.

Зачем нужна нормировка

Без знаменателя output является взвешенной суммой. При увеличении количества terms может изменяться типичный масштаб выходного вектора.

Это означает, что следующий слой получает значения, масштаб которых зависит от длины контекста.

Нормировка превращает сумму в аналог взвешенного среднего и уменьшает зависимость масштаба output от числа накопленных записей.

Это прежде всего вопрос численной устойчивости и масштаба активаций, а не гарантия качества модели.

Один оператор — три алгоритма вычисления

Для causal linear operator существуют три эквивалентные формы.

Literal parallel

$$Y = ((QK^{\top}) \odot M)V.$$

Характеристики:

- нет необходимости хранить recurrent state при вычислении полного известного input;
- существует global pairwise object $T \times T$;
- позиции хорошо параллелятся.

Recurrent

$$S_t = S_{t-1} + v_t k_t^{\top}, \quad y_t = S_t q_t.$$

Характеристики:

- pairwise matrix отсутствует;
- persistent state имеет размер $O(d_k d_v)$;
- позиции образуют последовательную цепочку длины T .

Chunkwise

Последовательность разбивается на blocks. Внутри блока interactions вычисляются parallel, а между blocks передаётся fixed-shape state.

Характеристики:

- persistent state $O(d_k d_v)$;
- вместо global $T \times T$ matrix существуют local $C \times C$ matrices;
- sequential depth между chunks примерно T/C .

Равенство outputs трёх форм является алгебраическим, а не приближённым: при одной и той же маске они вычисляют один mathematical operator.

Chunkwise form

Пусть block b начинается в позиции a_b . Состояние на входе содержит всё, что находится до текущего блока:

$$S_b^{\text{in}} = S_0 + \sum_{i < a_b} v_i k_i^\top.$$

Для позиции t внутри текущего блока output состоит из двух частей:

$$y_t = S_b^{\text{in}} q_t + \sum_{i=a_b}^t (q_t^\top k_i) v_i.$$

Первая часть представляет всё далёкое прошлое через fixed state.

Вторая часть представляет явные interactions внутри текущего блока.

Chunkwise form в матричной записи

Для текущего блока:

- Q_b — queries;
- K_b — keys;
- V_b — values;
- M_{c_b} — локальная causal mask;
- c_b — фактическая длина блока.

Output блока:

$$Y_b = Q_b (S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot M_{c_b}) V_b.$$

Первое слагаемое — contribution всех предыдущих chunks.

Второе — causal interactions текущего chunk.

Глобальная $T \times T$ matrix больше не нужна: pairwise object ограничивается размером блока.

Обновление state после chunk

Весь block можно свернуть одним матричным произведением:

$$V_b^\top K_b = \sum_{i \in I_b} v_i k_i^\top.$$

State на выходе:

$$S_b^{\text{out}} = S_b^{\text{in}} + V_b^\top K_b.$$

Через границу chunk передаётся только объект фиксированной формы $d_v \times d_k$.

Размер chunk как размен стоимости

Размер блока C управляет положением между двумя крайностями.

При большом C :

- больше pairwise interactions вычисляется локально;
- лучше parallelism внутри блока;
- local workspace и количество арифметики увеличиваются;
- число последовательных переходов между блоками уменьшается.

При маленьком C :

- local matrices меньше;
- больше информации проходит через recurrent state;
- количество последовательных стадий увеличивается.

Размер chunk поэтому является не просто «параметром скорости», а способом обменять одну координату стоимости на другую.

На практике подходящий chunk size зависит от hardware: local matrix желательно помещать в быструю память и обрабатываться эффективными матричными kernels.

Крайние случаи chunkwise form

Корректная реализация должна естественным образом обрабатывать крайние значения.

$$C = 1$$

Каждый chunk содержит один токен.

Chunkwise form вырождается в recurrent form.

$$C = T$$

Вся последовательность является одним chunk.

Chunkwise form вырождается в literal parallel form.

Неполный последний chunk

Если T не делится на C , последний block имеет меньший размер.

Добавленные padding-позиции не должны становиться реальными contributions в state или output.

Causality внутри chunk

Совместная обработка токенов одного блока не разрешает будущие interactions.

Local mask остаётся causal.

Отсутствие double counting

S_b^{in} должно содержать только предыдущие blocks.

Если сначала включить текущий block в state, а затем дополнительно посчитать его local interactions, его contribution будет учтён дважды.

Если, наоборот, преждевременно исключить нужный contribution, часть истории потеряется.

Арифметическая сложность трёх форм

Пусть C обозначает размер chunk.

Форма	Persistent state	Pairwise object	Sequential structure	Arithmetic scaling
Literal parallel	не нужен для полного input	$T \times T$	хорошо параллелится	$O(T^2(d_k + d_v))$
Recurrent	$O(d_k d_v)$	нет	$O(T)$ шагов	$O(T d_k d_v)$
Chunkwise	$O(d_k d_v)$	local $C \times C$	около $O(T/C)$ между chunks	$O(T d_k d_v + TC(d_k + d_v))$

Эти асимптотики не являются измерением wall-clock latency.

Recurrent form может выполнять меньше арифметики, но проигрывать parallel form на GPU из-за большого числа маленьких зависимых операций.

Большая matrix multiplication обычно гораздо эффективнее загружает массивно-параллельное оборудование.

Training и decode — разные режимы СТОИМОСТИ

Dense softmax на полной последовательности

Persistent context state не является главным ограничением.

Arithmetic scaling:

$$O(T^2 d).$$

Основная причина роста — pairwise interactions.

Softmax decode с KV-cache

Persistent context state:

$$O(Td).$$

На каждый новый token:

$$O(Td).$$

Ограничение — растущий KV-cache и объём его чтения.

Linear recurrent decode

Persistent state:

$$O(d_k d_v).$$

Стоимость нового token:

$$O(d_k d_v).$$

И память, и arithmetic cost одного шага не зависят от длины уже обработанного prefix.

Ограничением становится конечная информационная ёмкость state.

Linear recurrent training

Forward arithmetic:

$$O(Td_k d_v).$$

Математическое состояние имеет fixed shape, но это не означает constant training memory: backward требует хранить или восстанавливать промежуточные activations.

Кроме того, naive recurrent evaluation имеет последовательную зависимость длины T .

Linear chunkwise training

Arithmetic scaling:

$$O(Td_kd_v + TC(d_k + d_v)).$$

Ограничения:

- local interaction matrices;
- cross-chunk sequential stages;
- backward activations;
- аппаратная эффективность выбранного chunk size.

Chunkwise form возвращает parallelism внутри блока и оставляет последовательность только между блоками.

Prefill и autoregressive decode

Генерация фактически состоит из двух вычислительно разных фаз.

Обработка входного prefix — prefill

Весь prompt уже известен.

Позиции можно обрабатывать большими параллельными операциями. Режим ближе к training forward pass.

Основная нагрузка связана с большим объёмом арифметики.

Decode

Новые tokens появляются последовательно.

Следующая позиция неизвестна до получения предыдущей, поэтому parallelism по будущим позициям невозможен.

Для softmax attention каждый step должен читать растущий cache.

Для recurrent linear operator каждый step работает с fixed-shape state.

Поэтому ускорение одной архитектуры необходимо отдельно измерять для prefill и decode. Одно число «tokens/s» или «ускорение в N раз» без указания фазы может скрывать принципиально разные эффекты.

Почему меньше FLOPs не обязательно означает меньше времени

Пример с длиной $T = 1000$ и размерностями порядка 100 показывает различие между arithmetic complexity и hardware efficiency.

Parallel form создаёт порядка миллиона pairwise positions, но вычисляет их крупными matrix multiplications.

Recurrent form выполняет порядка тысячи последовательных обновлений матрицы размером 100×100 .

Даже если асимптотически recurrent form выгоднее, тысяча зависимых маленьких операций может выполняться медленнее одной большой матричной операции.

Отсюда два разных вопроса:

- сколько математической работы существует;
- насколько эффективно эта работа отображается на GPU.

Operator change и evaluation algorithm change

Необходимо различать два перехода.

Softmax kernel $\rightarrow$ factorized kernel

Меняется mathematical operator.

$$\kappa_{\text{sm}}(\hat{q}, \hat{k}) \longrightarrow \phi(\hat{q})^\top \phi(\hat{k}).$$

Модель начинает вычислять другую функцию.

Parallel ↔ recurrent ↔ chunkwise

Mathematical operator не меняется.

Меняется только evaluation algorithm.

Именно поэтому переход между тремя формами может быть точной алгебраической эквивалентностью, а переход от softmax к factorized kernel — нет.

Fixed state: постоянная цена, но не сохранность

Fixed-shape state решает конкретную проблему: размер persistent representation перестаёт зависеть от длины prefix.

Он не решает автоматически проблему памяти в информационном смысле.

Растущий KV-cache:

- выделяет каждой записи отдельное место;
- позволяет обращаться к отдельным прошлым keys/values;
- не смешивает их непосредственно в одном accumulator.

Fixed state:

- имеет постоянную форму;
- смешивает множество outer products;
- не позволяет в общем случае восстановить отдельные записи;
- подвержен interference;
- имеет конечную информационную ёмкость.

Поэтому правильный вывод не «модель теперь помнит бесконечный контекст за constant memory», а «цена хранения больше не растёт с prefix, потому что история заменена фиксированной свёрткой».

Направления развития fixed-state memory

Простое additive update

$$S_t = S_{t-1} + v_t k_t^\top$$

не управляет тем, что уже находится в памяти.

Из проблемы interference естественно возникают следующие классы механизмов:

- ослабление старого state перед новой записью;
- управляемое forgetting;
- проверка содержимого перед записью;
- адресная коррекция уже существующей записи;
- различная сила обновления отдельных компонент state.

То есть дальнейшие архитектуры можно рассматривать как варианты ответа на вопрос: **чем заменить безусловное сложение нового outer product с прошлым состоянием.**

Проверка математически эквивалентных реализаций

Совпадение parallel, recurrent и chunkwise forms не доказывает правильность реализованного оператора.

Все три формы могут исходить из одной и той же неверно понятой спецификации.

Пример — сдвиг causal mask.

Если во всех реализациях используется strict causality,

$$\tilde{y}_t = \sum_{i < t} (q_t^\top k_i) v_i,$$

то все три формы могут совпадать друг с другом практически до floating-point precision.

Но относительно требуемого inclusive operator

$$y_t = \sum_{i \leq t} (q_t^\top k_i) v_i$$

все три будут одинаково неправильными.

Согласованность реализаций проверяет в основном:

- ошибки индексации;
- ошибки перестановки осей;
- ошибки algebraic transformation;
- ошибки chunk aggregation.

Она не проверяет независимо исходное определение оператора.

Независимый reference implementation

Для проверки оптимизированной реализации полезен эталон, максимально близкий к математическому определению и максимально непохожий на оптимизированный алгоритм.

Для causal linear operator reference может буквально вычислять:

1. каждый t ;
2. все разрешённые $i \leq t$;
3. scalar product $q_t^\top k_i$;
4. умножение на v_i ;
5. прямое суммирование.

Эталону не требуется быть быстрым: он используется на маленьких tensors.

Чем меньше общих алгоритмических решений между reference и проверяемой реализацией, тем ниже вероятность общей систематической ошибки.

Если и reference, и оптимизированный вариант построены из одной и той же ошибочной маски или одного и того же преобразования, независимость проверки теряется.

Минимальные ручные тесты

Некоторые ошибки проще обнаружить не большим random test, а предельно маленьким примером.

Для $T = 1$ inclusive operator обязан вернуть

$$y_1 = (q_1^\top k_1)v_1.$$

Strict operator возвращает

$$\tilde{y}_1 = 0.$$

Такой тест непосредственно проверяет семантику диагонали и не зависит от сложности implementation.

Полезные крайние случаи для проверки chunkwise/recurrent кода:

- $T = 1$;
- chunk size $C = 1$;
- один chunk на всю последовательность;
- T не делится на C ;
- неполный последний chunk;
- $d_k \neq d_v$, чтобы случайно переставленные оси не скрывались квадратными shapes.

Floating-point error и критерий совпадения

Эквивалентные математические формы складывают одни и те же числа в разном порядке.

Floating-point addition не является идеально ассоциативным, поэтому побитовое совпадение не требуется.

Удобная метрика:

$$\Delta_{\max}(A, B) = \max_j |A_j - B_j|.$$

Для вычислений в FP64 очень маленькие расхождения порядка machine precision могут быть нормальны.

Поэтому проверять эквивалентность следует с небольшим численным tolerance, а не через точное равенство.

Одновременно tolerance не должен быть настолько большим, чтобы через него проходила семантическая ошибка вроде включающей и строгой causal masks.

Инструменты проверки в PyTorch

Для parallel form естественно использовать matrix multiplication QK^T , causal mask и последующее умножение на V .

Для inclusive causality нижнетреугольная mask должна включать диагональ. В PyTorch этому соответствует обычная нижнетреугольная маска.

Strict causality соответствует нижнему треугольнику со сдвигом на одну диагональ вниз.

Для recurrent form state удобно хранить непосредственно с shape $d_v \times d_k$ и обновлять outer product $v_t k_t^T$.

Для chunkwise form проверяются две независимые части:

- contribution carried state;
- local masked interactions.

Reference implementation полезно выполнять в FP64, даже если production implementation рассчитана на меньшую precision: это уменьшает влияние численной ошибки на сам эталон.

Максимальную абсолютную ошибку между формами следует рассматривать отдельно от ошибки относительно независимого reference:

- маленькая ошибка между тремя формами означает их согласованность;
- маленькая ошибка каждой формы относительно reference означает соответствие проверяемому определению.

Основные ошибочные выводы

«Контекст 100 000 токенов означает определённый объём памяти»

Нет. Длина является только одним множителем. Размер KV-cache зависит также от количества слоёв, distinct KV-heads, head dimension, dtype и batch size.

Модели с одинаковой context length могут радикально различаться по cache memory.

«Linear attention — быстрый способ вычислить softmax attention»

Нет, если softmax kernel заменён факторизуемым kernel. Это другой mathematical operator.

Отдельный класс методов действительно аппроксимирует softmax, но там возникает approximation error.

«Fixed-size state означает, что весь контекст сохранён»

Нет. Fixed-size означает только постоянное число хранимых scalars.

Записи superpose и могут интерферировать.

«Линейная асимптотика гарантирует меньшую latency»

Нет. FLOPs — только одна координата стоимости.

Большое хорошо параллелизуемое matrix multiplication может опережать меньшее число последовательных мелких операций.

«Если три формы совпали, реализация правильная»

Нет. Они могут одинаково реализовать один и тот же неверный operator.

Нужен независимый reference, выведенный непосредственно из specification.

«Постоянное состояние означает отсутствие forgetting»

Нет. KV-cache конструктивно хранит каждую запись отдельно. Fixed state, наоборот, вынужден уплотнять всё больше информации в объект неизменного размера.

«GQA/MQA — это настройка сжатия KV-cache уже обученной МНА-модели»

Нет. Число KV-heads является архитектурной характеристикой. Изменение этого числа меняет модель и обычно требует соответствующего обучения или специальной процедуры адаптации.

«Если существует recurrent state, обычный attention больше не нужен»

Нет. У подходов разные преимущества:

- softmax attention обеспечивает прямые pairwise interactions и хорошо параллелится на полном input;

- recurrent state обеспечивает fixed context cost на decode;
- chunkwise schemes позволяют комбинировать локальный parallelism и carried state.

Что читать дальше

Ashish Vaswani et al. — “Attention Is All You Need”, 2017.

[arXiv:1706.03762](#) Читать для исходной конструкции scaled dot-product attention, causal masking и multi-head attention, относительно которых определяются все дальнейшие изменения.

Noam Shazeer — “Fast Transformer Decoding: One Write-Head is All You Need”, 2019.

[arXiv:1911.02150](#) Читать ради Multi-Query Attention и мотивации через стоимость autoregressive decoding, размер KV-cache и memory bandwidth.

Joshua Ainslie et al. — “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”, 2023.

[arXiv:2305.13245](#) Читать для Grouped-Query Attention как промежуточной точки между MHA и MQA и для понимания размена между числом KV-heads и свойствами модели.

Angelos Katharopoulos et al. — “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”, 2020.

[arXiv:2006.16236](#) Главная работа для перехода от factorized attention kernel к recurrent fixed-size state и интерпретации causal linear attention как рекуррентного вычисления.

Krzysztof Choromanski et al. — “Rethinking Attention with Performers”, 2020.

[arXiv:2009.14794](#) Читать для противоположного подхода: не замены softmax другим kernel, а его приближения через random features. Полезно для чёткого различения operator replacement и approximation.

Imanol Schlag, Kazuki Irie, Jürgen Schmidhuber — “Linear Transformers Are Secretly Fast Weight Programmers”, 2021.

[arXiv:2102.11174](#) Читать ради интерпретации матрицы S_t как fast-weight memory, outer-product updates и понимания того, почему fixed-size associative memory сталкивается с interference.

Tri Dao et al. — “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, 2022.

[arXiv:2205.14135](#) Читать ради различия между FLOPs, workspace и HBM traffic: работа показывает, почему точный attention можно существенно ускорить, не меняя mathematical operator, за счёт организации движения данных.

Yutao Sun et al. — “Retentive Network: A Successor to Transformer for Large Language Models”, 2023.

[arXiv:2307.08621](#) Читать ради систематического использования parallel, recurrent и chunkwise recurrent representations одного sequence operator и понимания того, зачем practically нужны несколько вычислительных форм.